SIMATS ENGINEERING

ASSESSMENT TOOL 2 – INDUSTRY PROBLEM SOLVING

COURSE: ITA04STATISTICS WITH R PROGRAMMING

NAME: R. Sharmila

REG NO: 192521213

Q1 – Manufacturing: Defect Detection Syste

Aim: To write an R program to classify products based on their quality scores using for() loops, if-else, and next.

Algorithm:

1. Store the product quality scores.
2. Initialize counters for Accepted, Rework, and Rejected.
3. Use a for() loop to check each score.
4. Skip invalid scores using next.
5. Classify valid scores and update the counters.
6. Display the product status and summary report.

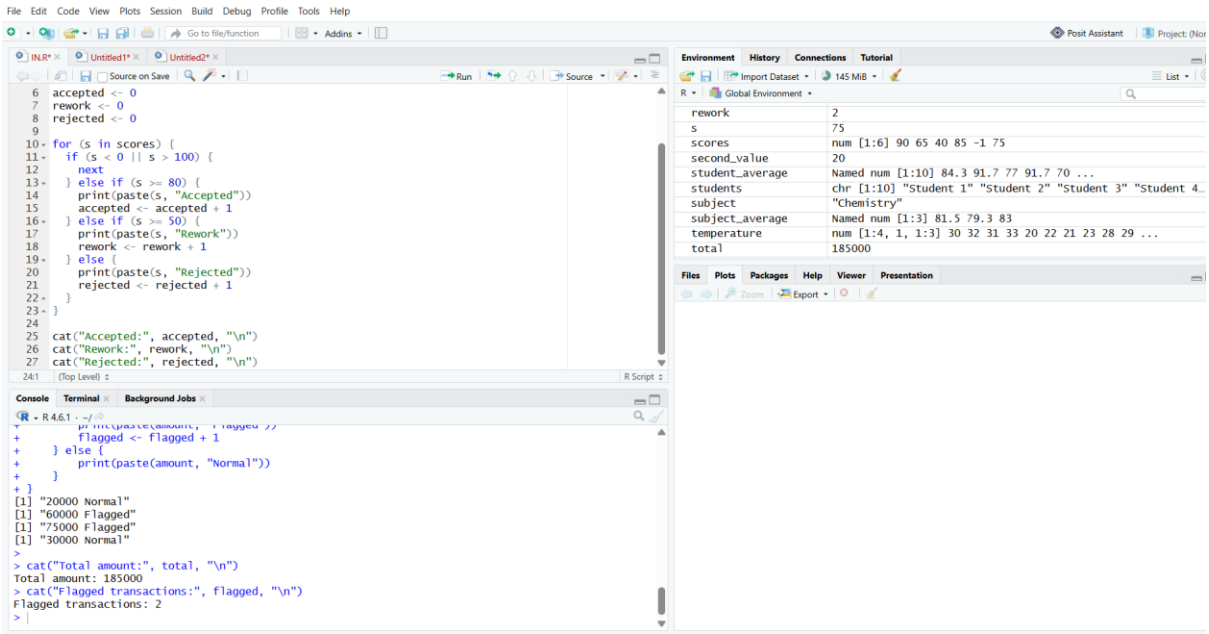
Program:

```
scores <- c(90, 65, 40, 85, -1, 75)
accepted <- 0
rework <- 0
rejected <- 0

for (s in scores) {
  if (s < 0 || s > 100) {
    next
  } else if (s >= 80) {
    print(paste(s, "Accepted"))
    accepted <- accepted + 1
  } else if (s >= 50) {
    print(paste(s, "Rework"))
    rework <- rework + 1
  } else {
    print(paste(s, "Rejected"))
    rejected <- rejected + 1
  }
}

cat("Accepted:", accepted, "\n")
cat("Rework:", rework, "\n")
```

cat("Rejected:", rejected, "\n")



Limitations:

1. The program processes only the given quality scores.
2. Invalid scores are skipped and not included in the report.
3. It cannot detect product defects automatically without quality score data.

Result: Thus, the R program successfully classified products as Accepted, Rework, and Rejected based on their quality scores and displayed the total count for each category.

Q2Banking: Fraud Transaction Monitoring

Aim: To write an R program to identify suspicious transactions exceeding ₹50,000 using a while() loop and conditional statements.

Algorithm:

1. Store transaction amounts.
2. Initialize the total amount and flagged transaction count.
3. Process transactions using a while() loop.
4. Stop processing when the amount is -1.
5. Skip other negative amounts using next.
6. Flag transactions above ₹50,000 and display the summary.

Program:

```
amounts <- c(20000, 60000, -5, 75000, 30000, -1)
i <- 1
total <- 0
flagged <- 0

while (i <= length(amounts)) {
  amount <- amounts[i]

  if (amount == -1) {
    break
```

```
  }

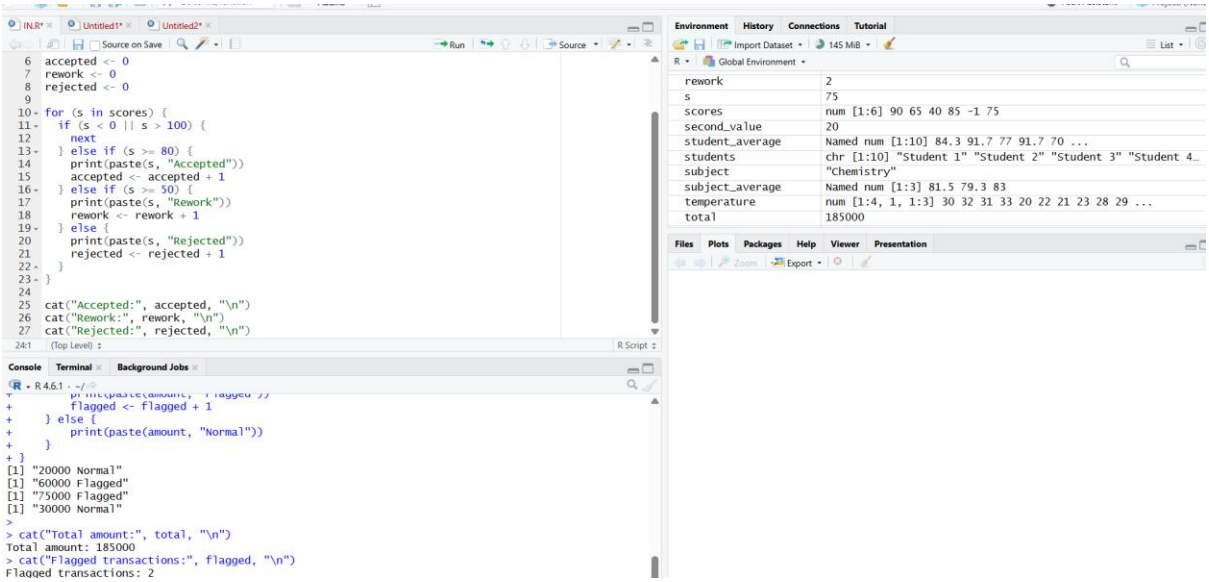
  i <- i + 1

  if (amount < 0) {
    next
  }

  total <- total + amount

  if (amount > 50000) {
    print(paste(amount, "Flagged"))
    flagged <- flagged + 1
  } else {
    print(paste(amount, "Normal"))
  }
}

cat("Total amount:", total, "\n")
cat("Flagged transactions:", flagged, "\n")
```



Limitations:

1. The program uses only transaction amounts to identify suspicious transactions.
2. Transactions below ₹50,000 may also be fraudulent but are not flagged.
3. Processing stops when the input value is -1.

Result: Thus, the R program successfully identified suspicious transactions exceeding ₹50,000, skipped invalid negative transactions, calculated the total transaction value, and displayed the number of flagged transactions.

Q3 – Retail: Inventory Replenishment System

Aim: To write an R program using loops, conditional statements, and a user-defined function with a default argument to manage product stock.

Algorithm:

1. Define a function to check product stock.
2. Set the default reorder limit to 10 units.
3. Classify stock as Reorder Required, Low Stock, or Sufficient Stock.
4. Store product names and stock quantities.
5. Use a loop to display the stock status of each product.
6. Display products that require replenishment.

Program:

```
check_stock <- function(name, qty, limit = 10) {
  if (qty < limit) {
    status <- "Reorder Required"
  } else if (qty <= 20) {
    status <- "Low Stock"
  } else {
    status <- "Sufficient Stock"
  }

  return(list(
```

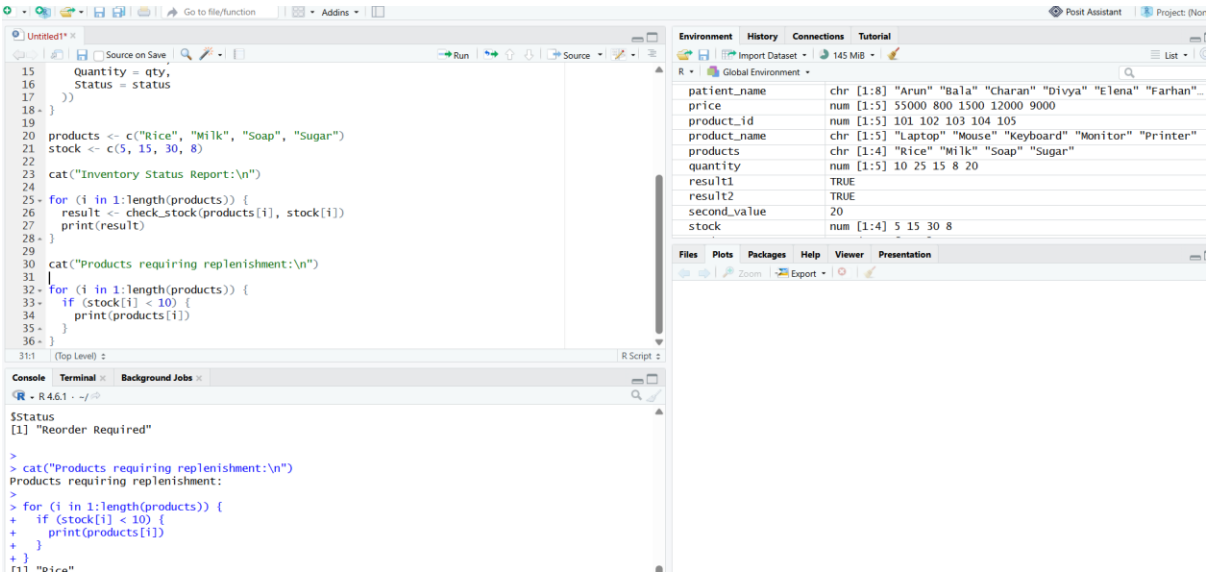
```
    Product = name,
    Quantity = qty,
    Status = status
  ))
}

products <- c("Rice", "Milk", "Soap", "Sugar")
stock <- c(5, 15, 30, 8)

for (i in 1:length(products)) {
  print(check_stock(products[i], stock[i]))
}

cat("Products requiring replenishment:\n")

for (i in 1:length(products)) {
  if (stock[i] < 10) {
    print(products[i])
  }
}
```



Limitations:

1. The program uses predefined product names and stock quantities.
2. The reorder limit is fixed at 10 units by default.
3. It does not automatically update inventory or place replenishment orders.

Result: Thus, the R program successfully classified products based on stock levels and identified products requiring replenishment using a user-defined function with a default argument.

Q4 – Logistics: Recursive Delivery Route Calculation

Aim: To calculate the total distance travelled by a delivery vehicle using recursion and iteration and compare both results.

Algorithm:

1. Store the distances of the route segments.
2. Check for negative distances.
3. Define a recursive function with a base case.
4. Calculate the total distance using recursion.
5. Calculate the total again using a for() loop.
6. Compare the two totals.

Program:

```
distance <- c(10, 20, 15, 25)

if (any(distance < 0)) {
  stop("Negative distances are not allowed")
}

recursive_sum <- function(x) {
  if (length(x) == 0) {
    return(0)
  }
```

```
  return(x[1] + recursive_sum(x[-1]))
}

total1 <- recursive_sum(distance)

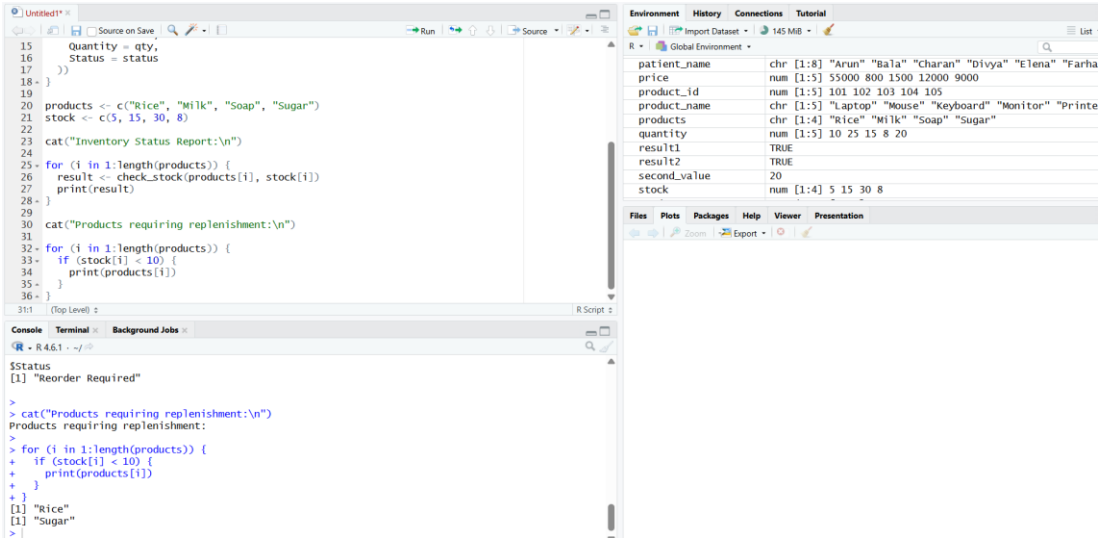
total2 <- 0
for (d in distance) {
  total2 <- total2 + d
}

cat("Recursive total:", total1, "km\n")
cat("Iterative total:", total2, "km\n")

if (total1 == total2) {
  cat("Both results are equal\n")
}
```

Output:

```
Recursive total: 70 km
Iterative total: 70 km
Both results are equal
```



Advantages and limitations:

- Recursion is easy to understand for smaller route lists but uses additional call-stack memory.
- Iteration generally uses less memory and is more suitable for a large number of route segments.

Result: Both recursive and iterative methods calculated the total distance as 70 km.

Q5 – Telecommunications: Network Failure Analysis

Aim: To identify high-latency network readings exceeding 200 milliseconds and detect three or more consecutive high-latency readings using iterative and recursive methods.

Algorithm:

1. Store the response-time readings.
2. Display readings exceeding 200 ms.
3. Define a recursive function to count consecutive high readings.
4. Use a loop to identify intervals containing three or more consecutive high readings.
5. Compare recursive and iterative results.
6. Display the findings.

Program:

```
time <- c(100, 250, 270, 230, 150, 220, 240, 100)


cat("High-latency readings:\n")
for (i in seq_along(time)) {
  if (time[i] > 200) {
    print(time[i])
  }
}
```

```
count <- 0
for (i in seq_along(time)) {
  if (time[i] > 200 && !is.na(time[i])) {
    count <- count + 1
  } else {
    count <- 0
  }

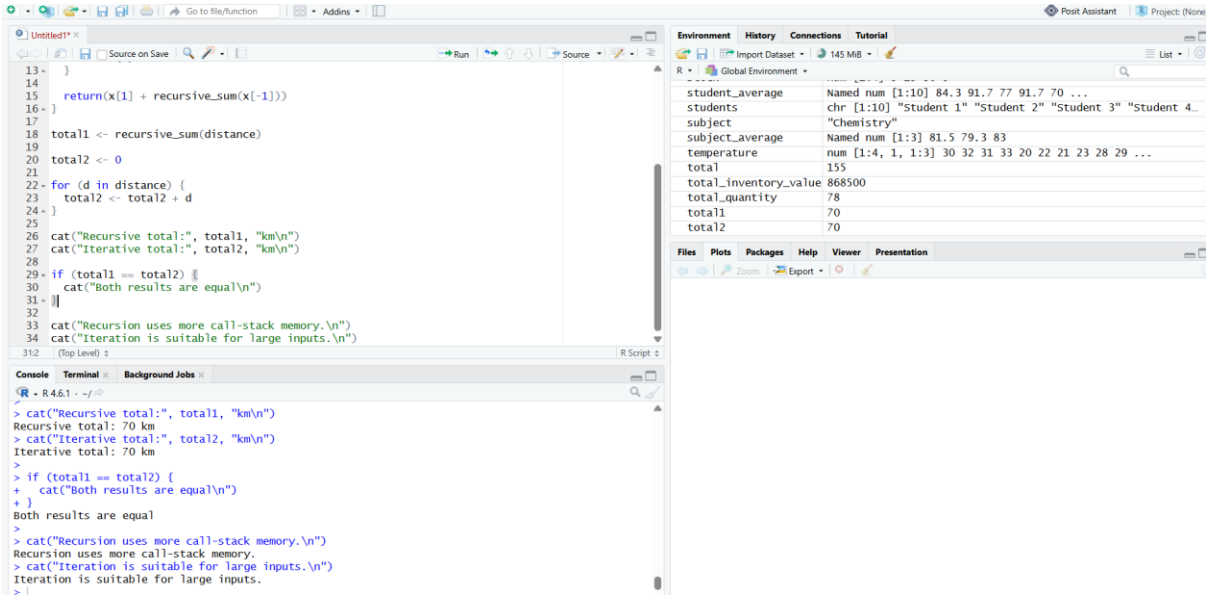
  if (count == 3) {
    cat("Three consecutive high readings detected\n")
  }
}

cat("Recursive count from position 2:",
    recursive_count(time, 2), "\n")

iter_count <- 0
for (i in 2:length(time)) {
  if (time[i] > 200 && time[i - 1] > 200) {
    iter_count <- iter_count + 1
  }
}

cat("Consecutive high-latency pairs:", iter_count, "\n")
```

Output:



Advantages and limitations:

- Recursion counts consecutive high readings from a specified position but may use more memory for long sequences.
- Iteration checks readings one by one and is generally more suitable for real-time monitoring.
-

Result: The program successfully identified high-latency readings and detected an interval with three consecutive high readings